

APÉNDICE A

DIAGRAMAS DE ARQUITECTURA

La Figura 9 resume el flujo jerárquico del sistema, desde la orquestación en C hasta la ejecución de los kernels escalares y vectoriales en los registros del procesador. Las flechas entre capas representan relaciones de invocación e implementación, no una secuencia temporal: los backends escalar y AVX2 son implementaciones alternativas de la misma interfaz. El controlador calcula $\sigma = \sqrt{\text{var}}$ después de obtener la varianza y antes de invocar la normalización.

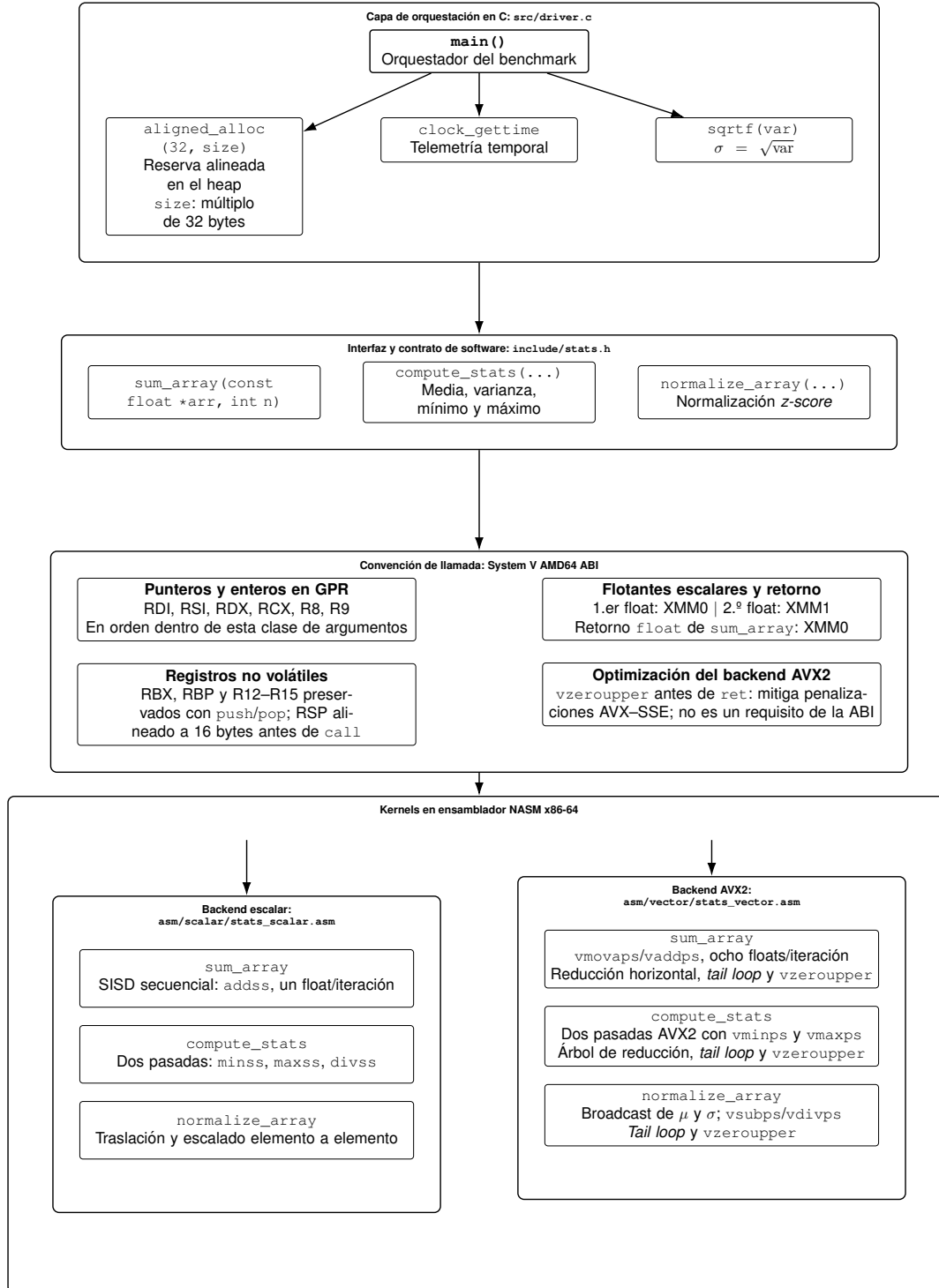


Figura 9: Flujo jerárquico desde la capa de orquestación en C hasta los backends escalar y AVX2 ejecutados por el hardware.

A-A. Bucle escalar SISD paso a paso

El kernel escalar recorre el arreglo de manera estrictamente secuencial. Cada iteración carga un único valor `float32`, lo acumula en el carril inferior de `xmm0` y avanza el índice una posición. Al no existir particionamiento vectorial, la rutina no necesita calcular un límite por bloques ni realizar una reducción horizontal al finalizar.

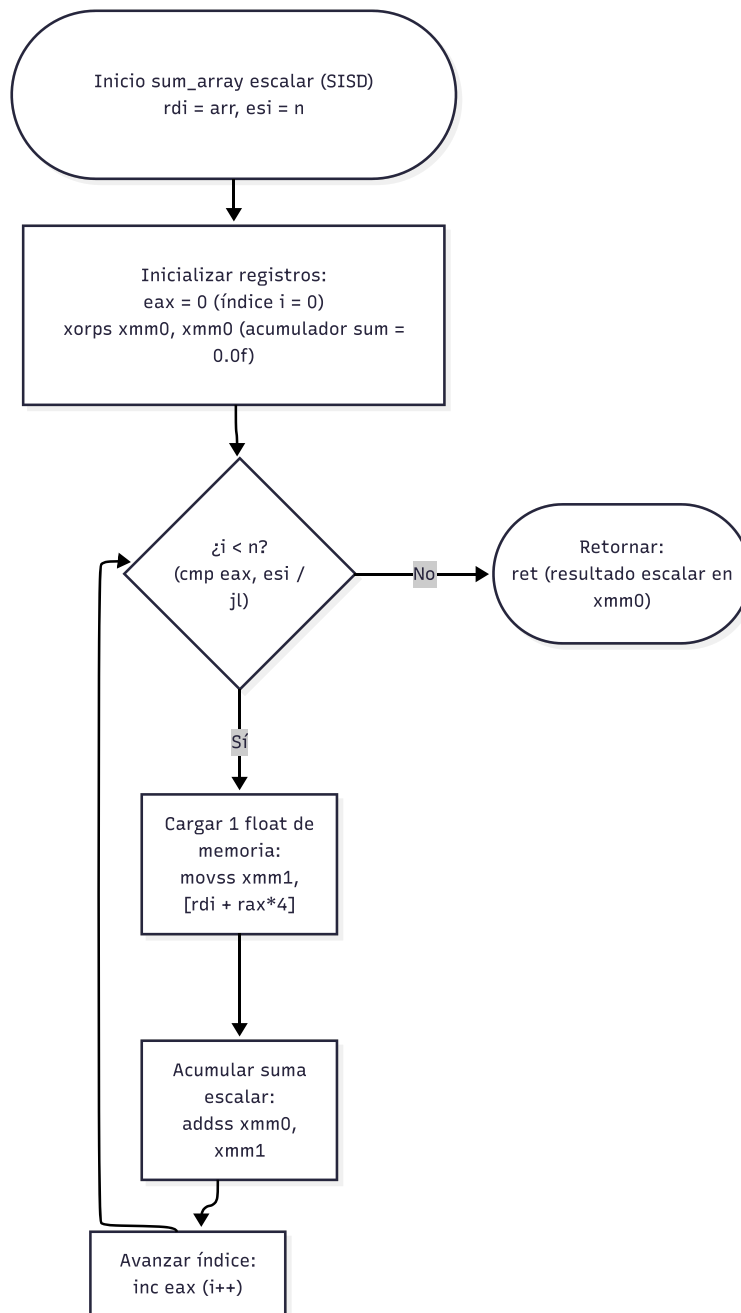


Figura 10: Diagrama del bucle escalar SISD paso a paso.

A-B. Bucle vectorial AVX2 con remanente

El kernel vectorial procesa ocho valores `float32` por iteración. Antes de entrar al bucle calcula $N_{\text{vec}} = n \& (\sim 7)$, reduce horizontalmente los acumuladores al concluir los bloques completos y procesa los $n \bmod 8$ elementos restantes mediante un bucle escalar de cierre.

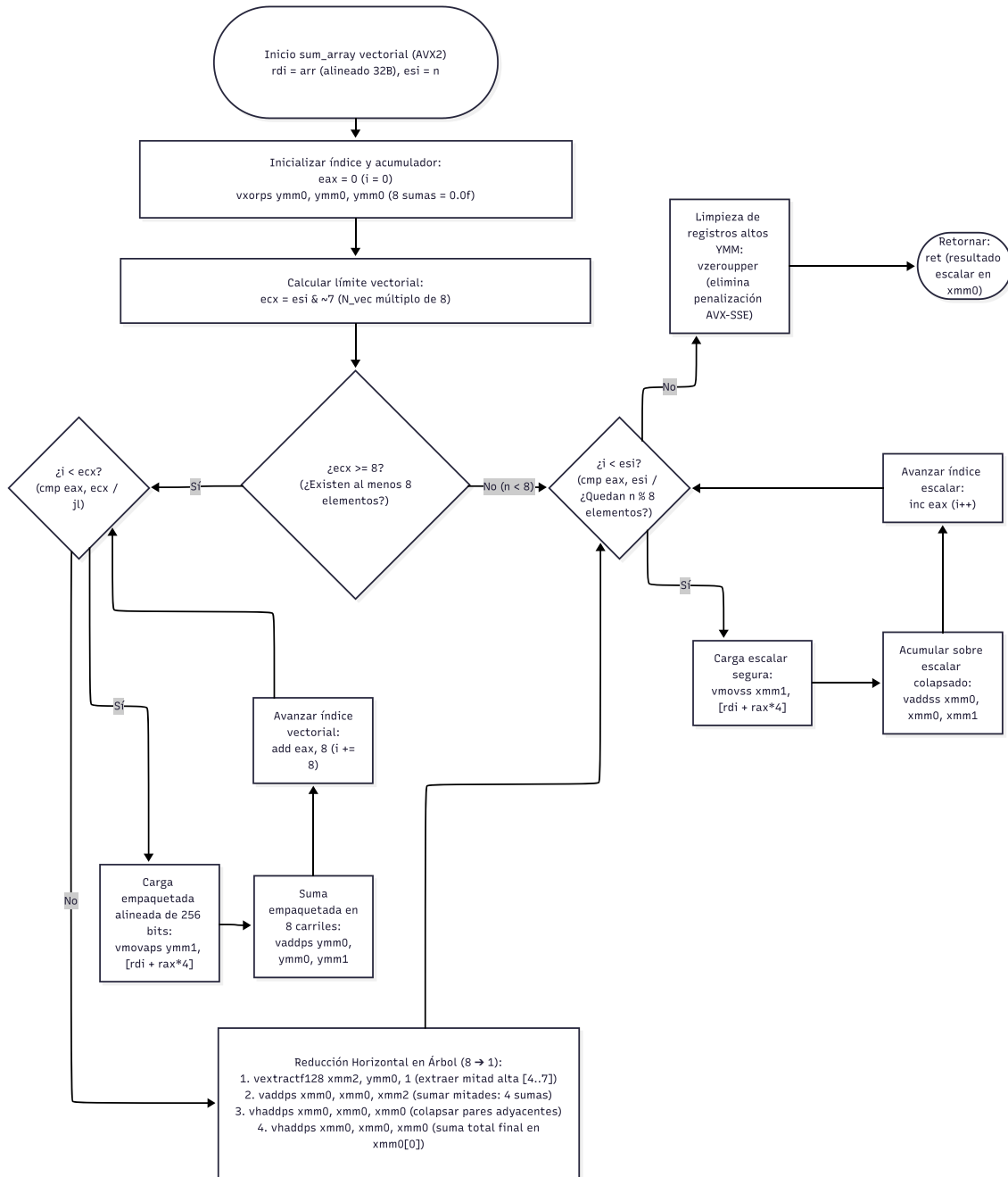


Figura 11: Diagrama del bucle Vectorial AVX2 con remanente paso a paso.

A-C. Normalización vectorial con guarda de desviación nula

La rutina `normalize_array` calcula

$$\text{out}[i] = \frac{\text{in}[i] - \mu}{\sigma}. \quad (29)$$

Los parámetros escalares μ y σ se replican en los ocho carriles de registros YMM mediante `vbroadcastss`. Antes de efectuar cualquier división, la rutina comprueba si $\sigma = 0,0$; en ese caso ejecuta una copia directa para evitar valores NaN o infinitos. La Figura ?? presenta ambas rutas y sus bucles escalares de remanente.

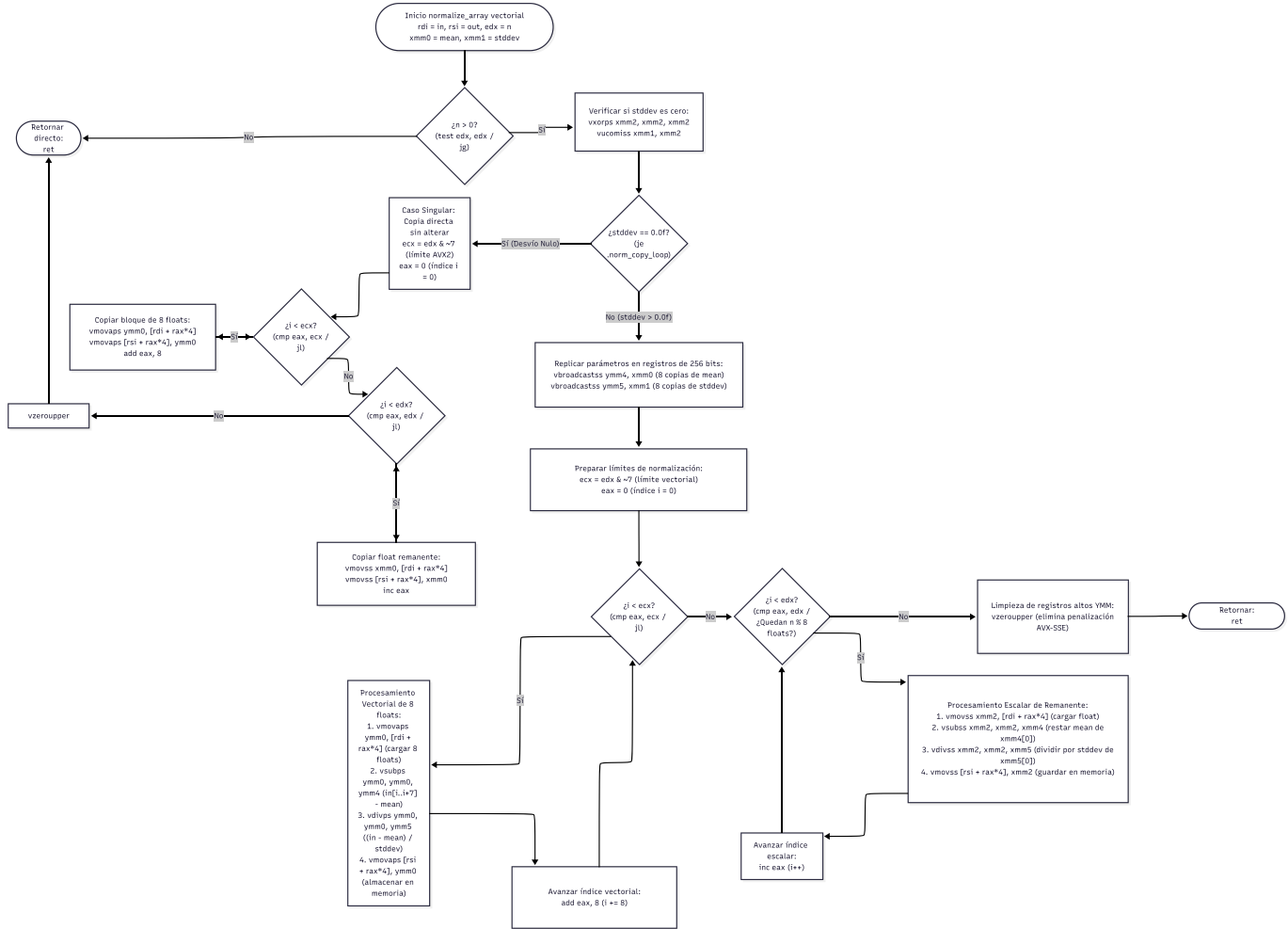


Figura 12: Flujo de Normalizar el Arreglo.

A-D. Flujo y validación de memoria

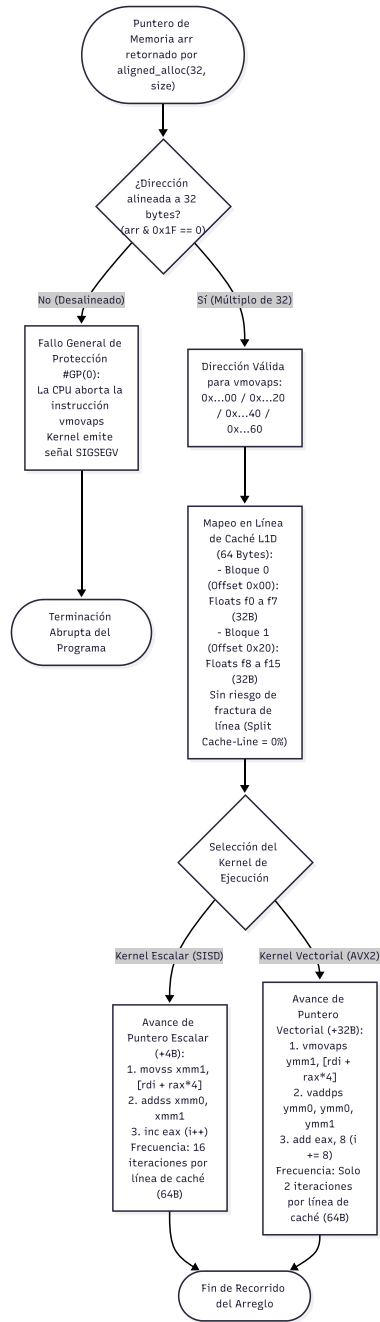


Figura 13: Flujo de validación de memoria

A-E. Diagrama 3: Tablas exhaustivas de asignación de registros

Cuadro XII: Convención de registros en System V AMD64 ABI.

Clase ABI	Registros de hardware	Regla de preservación	Comportamiento en subrutinas
Callee-saved (no volátiles)	RBX, RBP, R12–R15, RSP	Obligatorio preservar	Si la función los modifica, debe guardarlos en la pila y restaurarlos antes de retornar.
Caller-saved (volátiles)	RAX, RCX, RDX, RSI, RDI, R8–R11	Libre modificación	La función llamada puede utilizarlos sin preservación; el invocador asume que se destruyen tras un <code>call</code> .
SIMD volátiles	XMM0–XMM15 / YMM0–YMM15	Libre modificación	Todos los registros vectoriales son volátiles bajo la ABI System V AMD64.

Cuadro XIII: Asignación de registros de `sum_array`: implementación escalar.

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
RDI	GPR 64 bits	Caller-saved	Entrada/bucle	Puntero base al arreglo de entrada <code>arr</code> .
ESI	GPR 32 bits	Caller-saved	Entrada/bucle	Límite superior n y condición de parada.
EAX/RAX	GPR 32/64 bits	Caller-saved	Bucle escalar	Índice i ; direccionamiento <code>[rdi + rax*4]</code> .
XMM0	SSE 128 bits	Caller-saved	Bucle/retorno	Acumulador escalar inicializado con <code>xorps</code> ; valor de retorno en el carril 0.
XMM1	SSE 128 bits	Caller-saved	Bucle escalar	Carga temporal del elemento <code>arr[i]</code> mediante <code>movss</code> .

Cuadro XIV: Asignación de registros de `sum_array`: implementación AVX2.

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
RDI	GPR 64 bits	Caller-saved	Entrada/bucles	Puntero base <code>arr</code> , alineado a 32 bytes.
ESI	GPR 32 bits	Caller-saved	Entrada/remanente	Cantidad total de elementos n .
ECX	GPR 32 bits	Caller-saved	Particionado/AVX2	Límite $N_{vec} = n \& (\sim 7)$.
EAX/RAX	GPR 32/64 bits	Caller-saved	AVX2/remanente	Índice i ; avanza ocho elementos en AVX2 y uno en el remanente.
YMM0	AVX 256 bits	Caller-saved	Bucle AVX2	Acumulador de ocho sumas parciales, inicializado con <code>vxorps</code> .
YMM1	AVX 256 bits	Caller-saved	Bucle AVX2	Carga alineada de ocho valores mediante <code>vmovaps</code> .
XMM2	SSE 128 bits	Caller-saved	Reducción	Mitad alta de YMM0, extraída con <code>vextractf128</code> .
XMM0	SSE 128 bits	Caller-saved	Reducción/remanente/retorno	Acumulador colapsado con <code>vhaddps</code> ; recibe el remanente y retorna el resultado en el carril 0.
XMM1	SSE 128 bits	Caller-saved	Remanente escalar	Carga del valor restante mediante <code>vmovss</code> .

Cuadro XV: Asignación de registros de `compute_stats`: implementación escalar.

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
R12	GPR 64 bits	Callee-saved	Pasadas 1 y 2	Puntero base <code>arr</code> , recibido inicialmente en RDI.
R13D	GPR 32 bits	Callee-saved	Pasadas 1 y 2	Tamaño n , recibido inicialmente en ESI.
R14	GPR 64 bits	Callee-saved	Fin de pasada 1	Puntero <code>mean*</code> , recibido en RDX.
R15	GPR 64 bits	Callee-saved	Fin de pasada 2	Puntero <code>var*</code> , recibido en RCX.
RBX	GPR 64 bits	Callee-saved	Fin de pasada 1	Puntero <code>min*</code> , recibido en R8.
RBP	GPR 64 bits	Callee-saved	Fin de pasada 1	Puntero <code>max*</code> , recibido en R9.
EAX/RAX	GPR 32/64 bits	Caller-saved	Pasadas 1 y 2	Índice i : $1 \dots n - 1$ en P1 y $0 \dots n - 1$ en P2.
XMM0	SSE 128 bits	Caller-saved	Pasada 1	Acumulador de $\sum x_i$.
XMM1	SSE 128 bits	Caller-saved	Pasada 1	Acumulador del mínimo mediante <code>minss</code> .
XMM2	SSE 128 bits	Caller-saved	Pasada 1	Acumulador del máximo mediante <code>maxss</code> .
XMM3	SSE 128 bits	Caller-saved	Pasadas 1 y 2	Registro temporal para <code>arr[i]</code> y la desviación respecto a la media.
XMM4	SSE 128 bits	Caller-saved	Pasadas 1 y 2	Conversión de n a punto flotante mediante <code>cvtsi2ss</code> .
XMM5	SSE 128 bits	Caller-saved	Fin P1/P2	Media $\mu = \text{sum}/n$, conservada para la segunda pasada.
XMM6	SSE 128 bits	Caller-saved	Pasada 2	Acumulador de $\sum (x_i - \mu)^2$.

Cuadro XVI: Asignación de registros de `compute_stats`: implementación AVX2.

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
R12	GPR 64 bits	Callee-saved	Pasadas 1 y 2	Puntero base <code>arr</code> , alineado a 32 bytes.
R13D	GPR 32 bits	Callee-saved	Pasadas 1 y 2	Tamaño total n .
R14	GPR 64 bits	Callee-saved	Fin de pasada 1	Destino <code>mean*</code> .
R15	GPR 64 bits	Callee-saved	Fin de pasada 2	Destino <code>var*</code> .

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
RBX	GPR 64 bits	Callee-saved	Fin de pasada 1	Destino $\min^*$.
RBP	GPR 64 bits	Callee-saved	Fin de pasada 1	Destino $\max^*$.
ECX	GPR 32 bits	Callee-saved	Particionado P1/P2	Límite $N_{\text{vec}} = n \ \& \ (\sim 7)$.
EAX/RAX	GPR 32/64 bits	Callee-saved	Índice P1/P2	Avanza ocho elementos en AVX2 y uno en el remanente.
YMM0	AVX 256 bits	Callee-saved	P1/P2 vectoriales	P1: acumulador de suma; P2: acumulador de desviaciones cuadráticas.
YMM1	AVX 256 bits	Callee-saved	P1/P2 vectoriales	P1: acumulador mínimo; P2: carga y cálculo de $(x - \mu)^2$.
YMM2	AVX 256 bits	Callee-saved	P1 vectorial	Acumulador máximo mediante <code>vmaxps</code> .
YMM3	AVX 256 bits	Callee-saved	P1/P2 vectoriales	P1: carga de ocho valores; P2: difusión de la media.
XMM0	SSE 128 bits	Callee-saved	Reducciones P1/P2	Reducción de suma y de varianza.
XMM1	SSE 128 bits	Callee-saved	Reducción/remanente	Mínimo colapsado en P1 y temporal de diferencias en el remanente de P2.
XMM2	SSE 128 bits	Callee-saved	Reducción/remanente	Máximo colapsado y auxiliar escalar.
XMM3	SSE 128 bits	Callee-saved	Reducción/remanente	Auxiliar para <code>vshufps</code> y operaciones del remanente.
XMM4	SSE 128 bits	Callee-saved	Divisor P1/P2	Representación flotante de n mediante <code>vcvtsi2ss</code> .
XMM5	SSE 128 bits	Callee-saved	Pasadas 1 y 2	Media escalar μ , utilizada en la difusión y el remanente.

Cuadro XVII: Asignación de registros de `normalize_array`: implementación escalar.

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
RDI	GPR 64 bits	Callee-saved	Entrada/bucle	Puntero al arreglo de entrada <code>in</code> .
RSI	GPR 64 bits	Callee-saved	Entrada/bucle	Puntero al arreglo de salida <code>out</code> .
EDX	GPR 32 bits	Callee-saved	Entrada/bucle	Tamaño n .
XMM0	SSE 128 bits	Callee-saved	Entrada/bucle	Media escalar μ .
XMM1	SSE 128 bits	Callee-saved	Entrada/bucle	Desviación estándar σ .
XMM2	SSE 128 bits	Callee-saved	Detección de borde	Cero escalar para <code>ucomiss xmm1, xmm2</code> .
EAX/RAX	GPR 32/64 bits	Callee-saved	Bucle	Índice $i = 0 \dots n - 1$.
XMM3	SSE 128 bits	Callee-saved	Bucle	Temporal que contiene <code>in[i]</code> , <code>in[i] - μ</code> y el resultado normalizado.

Cuadro XVIII: Asignación de registros de `normalize_array`: implementación AVX2.

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
RDI	GPR 64 bits	Callee-saved	Entrada/bucles	Puntero base <code>in</code> , alineado a 32 bytes.
RSI	GPR 64 bits	Callee-saved	Entrada/bucles	Puntero base <code>out</code> , alineado a 32 bytes.
EDX	GPR 32 bits	Callee-saved	Entrada/bucles	Tamaño n .
XMM0	SSE 128 bits	Callee-saved	Entrada/difusión	Media escalar μ recibida desde <code>C</code> .
XMM1	SSE 128 bits	Callee-saved	Entrada/difusión	Desviación estándar σ recibida desde <code>C</code> .
XMM2	SSE 128 bits	Callee-saved	Detección/remanente	Comparación $\sigma = 0,0$ y temporal escalar.
YMM4	AVX 256 bits	Callee-saved	AVX2/remanente	Broadcast $[\mu, \mu, \mu, \mu, \mu, \mu, \mu, \mu]$.
YMM5	AVX 256 bits	Callee-saved	AVX2/remanente	Broadcast $[\sigma, \sigma, \sigma, \sigma, \sigma, \sigma, \sigma, \sigma]$.
ECX	GPR 32 bits	Callee-saved	Particionado AVX2	Límite $N_{\text{vec}} = n \ \& \ (\sim 7)$.
EAX/RAX	GPR 32/64 bits	Callee-saved	AVX2/remanente	Índice i ; paso de ocho en vector y uno en el remanente.
YMM0	AVX 256 bits	Callee-saved	Bucle AVX2	Carga de ocho valores, resta, división y almacenamiento empaquetados.